One Wide Feedforward is All You Need

Telmo Pessoa Pires[*†] **António V. Lopes** **Yannick Assogba** **Hendra Setiawan**[*]
Equall
telmo@equall.ai
Apple
{antoniovilarinholopes, yassogba, hendra}@apple.com

arXiv:2309.01826v2 [cs.CL] 21 Oct 2023

Abstract

The Transformer architecture has two main non-embedding components: Attention and the Feed Forward Network (FFN). Attention captures interdependencies between words regardless of their position, while the FFN nonlinearly transforms each input token independently. In this work we explore the role of the FFN, and find that despite taking up a significant fraction of the model's parameters, it is highly redundant. Concretely, we are able to substantially reduce the number of parameters with only a modest drop in accuracy by removing the FFN on the decoder layers and sharing a single FFN across the encoder. Finally we scale this architecture back to its original size by increasing the hidden dimension of the shared FFN, achieving substantial gains in both accuracy and latency with respect to the original Transformer Big.

1 Introduction

The Transformer architecture (Vaswani et al., 2017) has become the de facto paradigm in many Natural Language Processing (NLP) tasks, including Machine Translation (MT). Several studies have shown that Transformers exhibit impressive scaling-law properties (Gordon et al., 2021; Bansal et al., 2022; Ghorbani et al., 2022), wherein increasing the number of model parameters leads to further accuracy gains. In parallel with this architecture's impressive scaling of the numbers of parameters (Chowdhery et al., 2022), there is a growing trend towards reducing model footprints for real-world deployment, to satisfy practical constraints like latency requirements as well as memory and disk space limitations. In turn, researchers are actively exploring parameter sharing (Ge et al., 2022; Takase and Kiyono, 2023; Lou et al., 2022), reducing the dimensionality of Transformer components, and pruning components like attention heads (Voita et al., 2019; Michel et al., 2019).

Although the role of attention in learning pairwise dependencies between tokens is relatively well understood (Voita et al., 2019; Clark et al., 2019; Vig and Belinkov, 2019), the role of the Feed Forward Network (FFN) remains under-explored. Recently, Geva et al. (2021) established a connection between the FFN and attention by positing that the FFN corresponds to learnable *key-value* pairs where the weights of the first layer of the FFN corresponds to the *keys* and those of the second to the *values*. They find that the keys are able to capture salient textual patterns at each layer, and they notice that the classes of patterns tend to overlap between neighboring layers, indicating redundancy in the representation.

This observation motivates our work, where we revisit the conventional practice of allocating an individual FFN per layer. We investigate the effect of sharing and dropping the FFN across different layers on MT models. We conduct thorough experiments with different configurations of the Transformer, across different language pairs, including a low resource language pair and multilingual. In addition, we investigate the effect of the FFN in a decoder-only Transformer-based model. We find that a considerable level of redundancy exists between the encoder and decoder FFNs. As a result, we are able to eliminate the decoder FFN and share a single FFN across the encoder without significantly compromising the model's accuracy. This step leads not only to significant parameter savings but also opens up opportunities for further improvements. We also suggest using wider FFNs in the encoder while dropping the decoder's FFN, which results in a model with a similar size, but improved accuracy and reduced latency.

Finally we conduct a fine-grained analysis of the representational similarity between the original model, using one independent FFN per layer,

[*]Equal contribution.
[†]Work conducted while at Apple.

and various models with shared FFNs. Our results reveal that both model accuracy and the internal representation of Transformer blocks remain stable when sharing the FFN.

2 Background and Methodology

2.1 Transformer

The Transformer architecture has two main components: attention and the FFN, which are connected via a residual connection (He et al., 2016) and layer normalization (Ba et al., 2016). In an encoder-decoder model, there are two types of attention: self-attention and cross-attention. Self-attention is used in both the encoder and the decoder, allowing the model to focus on relevant information within the same sequence. Cross-attention is exclusive to the decoder and allows it to attend to the encoder's output. Attention takes as input a set of *queries*, *keys* and *values*, projected using four $\mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ matrices (one for the queries, keys, values, and final output) where d_{model} is the model's hidden dimension. It then applies the SOFTMAX function to allow it to focus on the most relevant values.

The FFN is applied after attention on both the encoder and the decoder and consists of the following 2-layer linear transformation:

$$\text{FFN}(\boldsymbol{x}) = \max(0, \boldsymbol{x}\mathbf{W}_1 + b_1)\mathbf{W}_2 + b_2, \quad (1)$$

where a RELU non-linearity is applied to the transformation of the input sequence ($\boldsymbol{x}$). At each layer, the FFN is parameterized with two matrices, $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ and $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ where d_{ff} is the *FFN dimension* and is usually set to $4 \times d_{\text{model}}$ (Vaswani et al., 2017).

Recent work has drawn a significant link between attention and the FFN (Geva et al., 2021), wherein $\mathbf{W}_1$ and $\mathbf{W}_2$ assume roles akin to the *keys* and *values* to an unnormalized attention where the input ($\boldsymbol{x}$) acts as the *query*. Unlike regular attention, the FFN employs a RELU, which allows *multiple* keys to significantly contribute to the final output (Geva et al., 2021). Additionally, these keys correspond to an inventory of salient patterns that are learned from the training data. Geva et al. (2021) suggest that at the lower layers the FFN learns shallow syntactic patterns and progressively learns deep semantic patterns on the deeper layers. Moreover, the authors find that there's a substantial overlap between patterns captured by adjacent layers, indicating that there are redundancies in the FFNs and suggesting a better allocation of these parameters might be beneficial for performance.

2.2 Sharing and Widening the FFN

The vanilla Transformer allocates one FFN for each layer of the encoder and decoder, i.e. FFN_i^{enc} or FFN_i^{dec}, respectively. Excluding embedding parameters, these FFNs occupy around two thirds of the parameter budget, while attention occupies the remaining third[1]. Earlier work found that constraining the parameterization of the decoder FFNs causes no degradation in accuracy (Ge et al., 2022). In this work, we share the parameters of the FFN across layers and/or across the encoder and decoder to minimize redundancy between FFNs.

Let N_{enc}, N_{dec} be the numbers of encoder and decoder layers, respectively. We consider multiple configurations for parameter sharing as follows:

- One $\mathbf{FFN}_{all}^{enc}$ for the whole encoder:

$$\text{FFN}_i^{enc}(\cdot) \overset{\text{tied}}{=} \text{FFN}_{all}^{enc}(\cdot), \forall i : 1 \leq i \leq N_{enc}$$

- One $\mathbf{FFN}_{all}^{dec}$ for the whole decoder:

$$\text{FFN}_j^{dec}(\cdot) \overset{\text{tied}}{=} \text{FFN}_{all}^{dec}(\cdot), \forall j : 1 \leq j \leq N_{dec}$$

- One $\mathbf{FFN}_{all}^{encdec}$ for both the encoder and the decoder:

$$\text{FFN}_i^{enc}(\cdot) \overset{\text{tied}}{=} \text{FFN}_j^{dec}(\cdot) \overset{\text{tied}}{=} \text{FFN}_{all}^{encdec}(\cdot),$$
$$\forall i, j : 1 \leq i \leq N_{enc}, 1 \leq j \leq N_{dec}$$

Additionally, we explore modifying the dimension of the shared FFN, which we denote as $d_{\text{ff}'}$. Setting $d_{\text{ff}'} > d_{\text{ff}}$ widens the shared FFN while $d_{\text{ff}'} < d_{\text{ff}}$ narrows it. We also consider the extreme cases of setting $d_{\text{ff}'}$ to 0 or to $(N_{enc} + N_{dec}) \times d_{\text{ff}}$ (and beyond). Setting $d_{\text{ff}'} = 0$ is equivalent to dropping the FFN[2] while setting $d_{\text{ff}'} = (N_{enc} + N_{dec}) \times d_{\text{ff}}$ is akin to sharing the concatenation of all individual FFNs.

Sharing the FFNs directly affects the number of parameters and, to a certain extent, latency. For instance, sharing FFN_{all}^{enc} for the whole encoder reduces the number of parameters by $(N_{enc} - 1) \times 2 \times d_{\text{model}} \times d'_{\text{ff}}$[3]; whereas removing the FFN on the

[1] Ignoring layer normalization, there are $4 \times d_{\text{model}} \times d_{\text{model}}$ parameters for attention vs $2 \times d_{\text{model}} \times d_{\text{ff}} = 8 \times d_{\text{model}} \times d_{\text{model}}$ parameters for the FFN, assuming $d_{\text{ff}} = 4 \times d_{\text{model}}$.

[2] In our experiments without the FFN (i.e., $d_{\text{ff}'} = 0$) we remove the residual connection and layer normalization associated with it, as they become redundant.

[3] Plus the layer normalization parameters, which we are ignoring for simplicity.

decoder, i.e., setting $d_{\text{ff}'} = 0$ for FFN^{dec}_{all}, reduces the parameters by $(N_{dec}) \times 2 \times d_{\text{model}} \times d'_{\text{ff}}$ and reduces the amount of computation to be done. This is particularly important during inference since the forward pass of the decoder is autoregressive, and changing the decoder's FFN dimension has a higher latency impact than on the encoder.

Since different configurations have different impacts, we analyse the trade-off between model size, latency, and accuracy: (i) How many parameters can be shared/pruned with negligible (if any) accuracy degradation? (ii) Are the encoder and decoder FFNs affected similarly? (iii) Keeping the same model size, can the FFN parameters be allocated more efficiently?

We propose a novel configuration, which we call the *One Wide FFN* model, consisting of a single shared wide FFN on the encoder and no FFN on the decoder. To keep the number of parameters the same as in the baseline, we increase the shared FFN dimension accordingly: FFN^{enc}_{all} with $d_{\text{ff}'} = (N_{enc} + N_{dec}) \times d_{\text{ff}}$.

For completeness, we include similar experiments on the attention mechanism in Appendix B. These experiments show that, contrary to the FFN, individual layer-specific attention weights are more important and not as redundant, as sharing the attention leads to significant accuracy drops.

2.3 Representational Similarity

Besides investigating the impact on accuracy, we study the similarity between different models in terms of their *internal representations* and the *semantic space* they produce.

We use Linear Centered Kernel Alignment (CKA, Kornblith et al., 2019) to measure the similarity between the internal representations of different models. CKA uses inner products to estimate how similar the kernel matrices of two different representations are, and is based on the Hilbert-Schmidt Independence Criterion (HSIC, Gretton et al., 2005), a statistical measure of independence of two random variables. Linear CKA uses the dot product as a kernel and can be written as:

$$\text{CKA}(\mathbf{A}, \mathbf{B}) = \frac{||\mathbf{A}\mathbf{B}^{\text{T}}||_{\text{F}}^2}{||\mathbf{A}^{\text{T}}\mathbf{A}||_{\text{F}}||\mathbf{B}^{\text{T}}\mathbf{B}||_{\text{F}}},$$

where $||\cdot||_{\text{F}}$ is the Frobenius norm while $\mathbf{A}$ and $\mathbf{B}$ are mean-centered (i.e., we subtract the mean) feature matrices of the layers under comparison, computed on the same dataset. Both matrices are $n \times d$, where n is the number of sentences in the dataset and d is the output dimension of the component, and are obtained by averaging the activation of all tokens in each sentence[4]. The linear kernel is straightforward to compute and Kornblith et al., 2019 report strong empirical performance of linear CKA compared to other kernels and methods.

To measure the similarity between the semantic spaces of different models, we use Local Neighborhood Similarity (LNS, Boggust et al., 2022). Local neighborhood similarities have been previously been used in analyzing semantic shifts in word embeddings (Hamilton et al., 2016). The premise of LNS is that two semantic spaces are similar if a sentence has similar neighbors in the two spaces. The LNS of a sentence s between models 1 and 2 is defined as:

$$\text{LNS}(s) = Sim(k\text{-NN}_1(s),\ k\text{-NN}_2(s)),$$

where k-NN(s) is the set of k nearest neighbors of sentence s for a model and Sim is the intersection-over-union (Jaccard similarity) of the two sets of neighbors. For each pair of components (attention and FFN) in models 1 and 2 we compute the LNS of all sentences in the evaluation dataset and take the mean LNS as our layer similarity measure. The smaller the value of k the more local the neighborhoods we are comparing, and the more specific the retrieval task. We pick k to be small enough to visually inspect sentence neighborhoods if necessary. In our analysis, we use cosine distance as the distance metric between activations and set k to 5% of the dataset size (~ 100 sentences).

3 Experimental Setup

Data In our experiments, we show results on WMT22 English (EN) $\rightarrow$ German (DE) (296M pairs), which we obtained using the provided `mt-data` scripts[5], WMT16 EN $\rightarrow$ Romanian (RO) (610K pairs), and for the multilingual setup of Pires et al. (2023), consisting of 10 languages: German, English, Spanish, French, Italian, Japanese, Korean, Portuguese, Swahili, and Chinese. In our analysis, we mostly focus on WMT22 EN $\rightarrow$DE.

Following Schmidt et al. (2022), we use WMT'16 provided scripts to normalize the RO side. EN $\rightarrow$RO keeps diacritics for producing accurate translations. For more details refer to Schmidt et al.

[4]We use the source sentence and force decode the first reference to compute the encoder and decoder representations, respectively.

[5]https://www.statmt.org/wmt22/mtdata/